

Cursor SOTA para o POC Buddemeyer

Manual completo de Skills, Rules, Hooks, Subagentes, simulação, governança e prompts estruturados para o supervisor de pick-and-place em Curitiba.

13	3	2	1
Skills	Rules	Hooks	Verificador

Preparado para Guilherme e equipe técnica do POC de Curitiba
Arquitetura revisada em 8 de setembro de 2026

Sumário

1. Visão executiva	3
2. Conceitos fundamentais	4
3. Arquitetura instalada	6
4. As 13 skills em detalhe	7
5. As 3 rules	16
6. Hooks e controles determinísticos	17
7. Subagente industrial-change-verifier	18
8. Documentos que tornam o Cursor especialista	19
9. Instalação e atualização no POC	19
10. Prompts estruturados para o chat	21
11. Roteamento automático esperado	24
12. Por que esta arquitetura está no SOTA	26
13. Governança e fluxo diário	27
14. Glossário técnico	28
15. Referências oficiais	29

Como ler este manual

Comece pelos capítulos 1 a 3 para compreender a arquitetura. Use o capítulo 4 como referência das skills, os capítulos 9 e 10 para instalar e operar, e os capítulos 12 e 13 para governança e evolução.

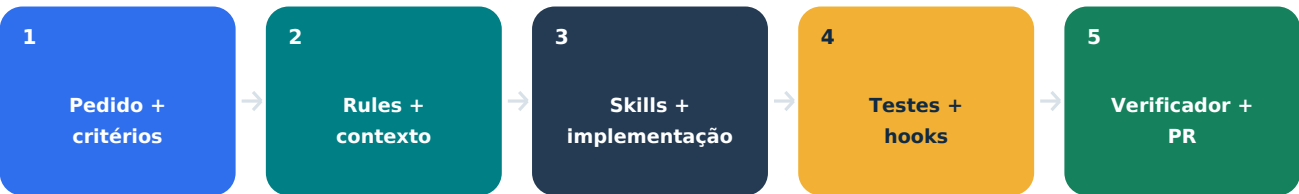
1. Visão executiva

SOTA em uma frase

O Cursor deixa de ser apenas um gerador de código e passa a operar dentro de um sistema de engenharia: contexto permanente, conhecimento especializado sob demanda, verificações determinísticas, simulação, revisão independente, Git e aprovação humana.

Este manual explica o pacote `cursor-sota-buddemeyer-poc-curitiba.zip`. Ele foi projetado para o supervisor de pick-and-place que integra visão computacional, calibração câmera-robô, aplicação Python/PySide6, CLP Omron NX102, CIP sobre EtherNet/IP, robô e plataforma de ventosas. Onde a tecnologia, a versão ou o caminho real ainda não estiver confirmado, o pacote ordena ao Agent que marque o dado como não verificado em vez de inventá-lo.

A ideia central é defesa em profundidade. Uma rule não executa teste; uma skill não é uma barreira de safety; um hook não compreende toda a planta; um subagente não substitui o responsável local. As camadas se complementam porque cada uma reduz uma classe diferente de erro.



Defesa em profundidade: cada camada reduz uma classe diferente de erro

Capacidade	Quantidade	Papel principal
Agent Skills	13	Workflows especializados acionados por contexto ou por /nome-da-skill.
Project Rules	3	Invariantes e padrões carregados de forma permanente ou inteligente.
Hooks	2	Gate de comandos arriscados e validação sintática pós-edição.
Subagente	1	Revisão independente, em contexto separado e somente leitura.
Documentos de projeto	3	Fatos locais, mapa de código e contrato da FSM/interfaces.
Biblioteca de prompts	1	Pedidos repetíveis para feature, customização, problema e revisão.

Limite de segurança

A instalação não concede autoridade para forçar I/O, transferir projeto, sincronizar NX102, comandar robô, alterar safety, firmware ou rede de produção. Qualquer ação física continua sujeita a procedimento, permissivos, responsável local, backup, janela e rollback.

2. Conceitos fundamentais

Os termos abaixo possuem papéis diferentes. Misturá-los produz prompts longos, controles fracos e dificuldade de manutenção.

Conceito	Definição operacional
Agent	Modelo que interpreta o pedido, explora o repositório, usa ferramentas e propõe ou realiza mudanças dentro das permissões.
Skill	Pasta com SKILL.md que ensina um workflow especializado. O Cursor a descobre e a carrega quando a descrição combina com o pedido.
Rule	Instrução de nível de projeto em .cursor/rules/*.mdc. Pode ser permanente, inteligente, associada a arquivos ou manual.
Hook	Processo externo disparado por eventos do Agent. Recebe JSON por stdin, devolve JSON por stdout e pode observar, validar ou pedir permissão.
Subagente	Agente especializado com contexto separado. O verificador do pacote usa readonly: true para não editar o que está auditando.
Prompt	Especificação do trabalho: objetivo, contexto, critérios de aceite, restrições, evidências e formato da entrega.
Plan	Fase em que o Agent inspeciona e propõe o mapa de impacto antes de alterar código.
Context docs	Arquivos versionados que registram fatos estáveis do POC e evitam alucinações sobre caminhos, tags e contratos.
CI	Execução automatizada e repetível de lint, tipos, testes e checks em cada mudança. Complementa os hooks locais.
HIL	Hardware-in-the-loop: teste controlado com hardware real ou representativo, separado da simulação e da produção.

2.1 Skills

Cada skill reside em `.cursor/skills/<categoria>/<nome>/SKILL.md`. O frontmatter declara name e description; a descrição é o principal sinal para o roteamento automático. O corpo só é carregado quando a skill se torna relevante, aplicando divulgação progressiva de contexto.

- Acionamento automático: o Agent compara o pedido com as descrições disponíveis.
- Acionamento explícito: Guilherme digita `/nome-da-skill` no chat.
- Escopo: categorias são organizacionais; a identidade é a pasta imediatamente acima de SKILL.md.
- Boa prática: uma responsabilidade clara por skill e descrições discriminantes, sem duplicação genérica.

Skill não é memória mágica

Ela fornece método e conhecimento do projeto, mas ainda depende dos arquivos reais, das evidências e dos testes. Por isso o pacote inclui CODEBASE_MAP.md e contratos a preencher.

2.2 Rules

Rules são instruções de nível de sistema do projeto. Vivem em `.cursor/rules/*.mdc` e controlam como o Agent deve interpretar ou modificar o repositório. Uma rule com `alwaysApply: true` entra em toda conversa; uma rule com descrição e `alwaysApply: false` pode ser selecionada inteligentemente.

Rule	Comportamento	Uso neste pacote
Sempre aplicada	Presente em toda sessão.	Contexto do POC, evidência e limites de segurança.
Aplicação inteligente	Incluída quando a descrição é relevante.	Arquitetura PySide6 e adaptadores de hardware.
Por arquivos	Ativada por globs confirmados.	Pode ser adotada depois que CODEBASE_MAP.md tiver os paths reais.
Manual	Somente quando mencionada.	Não utilizada como padrão do pacote.

2.3 Hooks - o termo correto, não 'looks'

Neste contexto, o termo correto é hooks. Um hook é um gancho de execução: o Cursor chama um script antes ou depois de determinado evento. A grafia 'looks' aparenta ser apenas um lapso de digitação. O manifesto de projeto fica em `.cursor/hooks.json`.

- Determinístico: uma regra de código valida sintaxe ou reconhece um padrão sem depender do julgamento do LLM.
- Event-driven: pode executar antes de shell, depois de editar arquivo e em outros pontos do ciclo do Agent.
- Controlável: hooks pré-execução podem retornar allow, deny ou ask conforme o evento suportado.
- Limitado: um regex não conhece toda ferramenta industrial e nunca substitui safety ou revisão humana.

2.4 Subagente e revisão independente

Um subagente recebe uma tarefa focada e trabalha em contexto separado. Isso reduz a tendência de quem implementou justificar a própria solução. O arquivo `industrial-change-verifier.md` define um revisor com `readonly: true`, impedindo edições e comandos de mudança de estado durante a auditoria.

2.5 Simulador, fake, mock e HIL

Termo	Uso recomendado
Fake	Implementação simples e funcional da mesma interface, por exemplo um PLC fake com estado em memória.
Mock	Objeto que verifica interações esperadas; útil com moderação para não acoplar o teste a detalhes internos.
Simulador	Modelo comportamental capaz de reproduzir tempo, ACK, timeout, falha, reconexão e sequências.
Record/replay	Reexecução de eventos sanitizados e versionados; não deve carregar segredos nem tornar o teste dependente de ruído.
HIL	Etapa controlada com hardware. Confirma integração física, mas não elimina a necessidade de testes offline.

3. Arquitetura instalada

O ZIP não possui pasta externa. Ao ser extraído na raiz do repositório, cria diretamente a seguinte estrutura:

```
<raiz-do-repositório>/
|-- .cursor/
|   |-- hooks.json
|   |-- agents/industrial-change-verifier.md
|   |-- hooks/
|       |-- guard-industrial-commands.py
|       |-- validate-edited-file.py
|   |-- rules/
|       |-- 00-poc-context-and-safety.mdc
|       |-- hardware-adapters.mdc
|       |-- pside-ui-architecture.mdc
|   |-- skills/<categoria>/<skill>/SKILL.md
|-- docs/
|   |-- cursor-agent/
|       |-- ARQUITETURA_SOTA_CURSOR.md
|       |-- AVALIACAO_ARQUIVOS_E_ESTRUTURA.md
|       |-- PROMPTS_CURSOR_POC.md
|   |-- project/
|       |-- CODEBASE_MAP.md
|       |-- INTERFACES_AND_STATE_MACHINE.md
|       |-- POC_CURITIBA_CONTEXT.md
|-- 00_LEIA_PRIMEIRO.md
|-- TUTORIAL_INSTALACAO_E_SKILLS.md
```

3.1 Separação de responsabilidades

Camada	Local	Responsabilidade	Não substitui
Contexto permanente	.cursor/rules/00-poc-context-and-safety.mdc	Invariantes, evidência e limites.	Safety e procedimento.
Conhecimento sob demanda	.cursor/skills/**/SKILL.md	Método especializado por intenção.	Código e teste.
Controles objetivos	.cursor/hooks.json e .cursor/hooks/	Gate e checks rápidos.	CI e intertravamento.
Revisão independente	.cursor/agents/industrial-change-verifier.md	Auditoria somente leitura.	Aprovador humano.
Fatos locais	docs/project/	Paths, versões, sinais, FSM e comandos.	Fonte primária atual.
Uso repetível	docs/cursor-agent/PROMPTS_CURSOR_POC.md	Templates de pedidos e aceite.	Decisão de engenharia.

3.2 Fluxo de uma mudança

- Guilherme descreve objetivo, contexto, critérios de aceite e restrições.
- A rule permanente injeta os invariantes do POC; rules inteligentes entram conforme a camada.
- O Agent seleciona uma ou mais skills e começa em Plan para mapear impacto.
- A implementação usa interfaces, simuladores e testes antes de qualquer HIL.
- Hooks avaliam comandos suspeitos e sintaxe básica dos arquivos editados.
- O subagente readonly revisa diff, testes, FSM, concorrência, freshness e rollback.
- O time revisa o PR e decide sobre bancada, HIL e liberação local.

4. As 13 skills em detalhe

As skills foram divididas para melhorar o roteamento automático e reduzir contexto desnecessário. Um pedido pode acionar mais de uma skill: por exemplo, uma nova tela de receitas combina customização, PySide6, configuração e simuladores.

Grupo	Skills
Mudança do aplicativo	customize-buddemeyer-supervisory; develop-pyside6-supervisory-ui; manage-buddemeyer-recipes-config; package-release-buddemeyer
Teste e confiabilidade	test-buddemeyer-with-simulators; debug-startup-restart; harden-buddemeyer-24x7
Processo e percepção	validate-pick-place-cycle; validate-vision-inference; validate-camera-robot-calibration
Diagnóstico industrial	incident-triage-buddemeyer; debug-nx102; debug-cip-ethernet-ip

4.1. customize-buddemeyer-supervisory

Finalidade

Porta de entrada para adicionar features ou alterar comportamento do supervisório como uma mudança de produto industrial, e não como uma edição isolada.

Quando deve ser acionada. Nova tela, caso de uso, regra funcional, integração ou fluxo que atravessa mais de uma camada do aplicativo.

Entradas e evidências. Resultado operacional desejado, critérios mensuráveis, não objetivos, estados da FSM, usuários afetados, entrypoints, chamadores, contratos, configuração e testes encontrados no repositório.

Fluxo orientado pela skill

- Reescrever o pedido em resultado operacional e critérios observáveis.
- Inspecionar POC_CURITIBA_CONTEXT.md, CODEBASE_MAP.md e INTERFACES_AND_STATE_MACHINE.md.
- Tracar UI > aplicação > domínio/FSM > adaptadores e produzir mapa de impacto.
- Propor a menor mudança reversível e preservar compatibilidade ou explicitar migração.
- Aplicar timeout, cancelamento, idempotência, identidade e validade dos dados quando pertinentes.
- Criar testes que falham antes e passam depois; executar primeiro offline.
- Acionar industrial-change-verifier antes de considerar a mudança pronta.

Saída esperada. Resumo antes/depois, arquivos alterados, decisão arquitetural, testes e resultados literais, riscos residuais, itens não verificados e rollback.

Por que representa SOTA. Fecha o ciclo completo requisito > arquitetura > implementação > teste > verificação > rollback, reduzindo mudanças pontuais sem contrato.

Exemplo de prompt

Adicione [FEATURE] para [RESULTADO]. Comece em Plan, descubra o fluxo e os contratos reais, apresente o mapa de impacto e implemente a menor mudança reversível. Aceite quando: [CRITÉRIOS]. Não acesse hardware. Teste unidade + Qt + integração simulada, use o verificador e entregue diff, resultados, riscos e rollback.

4.2. develop-pyside6-supervisory-ui

Finalidade

Orienta telas, widgets, dialogs, sinais, slots, workers, timers e lifecycle da interface PySide6/Qt.

Quando deve ser acionada. Tela nova, UI congelando, atualização assíncrona, callback duplicado, thread residual, fechamento incompleto ou estados visuais inconsistentes.

Entradas e evidências. Versão real do PySide6/Qt, fonte da UI (.ui, Python ou QML), padrão arquitetural, widgets afetados, sinais/slots, workers, timers e testes existentes.

Fluxo orientado pela skill

- Manter widgets como apresentação; regras críticas ficam no domínio/FSM.
- Nunca executar câmera, inferência, CIP, polling ou sleep na thread da UI.
- Atualizar widgets na thread da UI por Signal/Slot e conexões apropriadas.
- Definir ownership, cancelamento, timeout e cleanup de worker, timer e callback.
- Tornar start, stop e close idempotentes e resistentes a reinicializações.
- Testar estados, sinais, erros e encerramento com QSignalSpy ou ferramenta já adotada.

Saída esperada. Fluxo visual antes/depois, responsabilidades por camada, política de thread e cleanup, testes Qt, inspeção visual, risco e rollback.

Por que representa SOTA. Combina padrões oficiais do Qt com isolamento do hardware e testes de lifecycle, pontos frequentes de regressão em supervisórios.

Exemplo de prompt

A tela congela durante [OPERAÇÃO]. Localize chamadas bloqueantes, sinais, slots, workers e timers. Preserve o event loop, defina cancelamento e cleanup, e teste abrir/fechar e Start/Stop repetidos sem hardware real.

4.3. test-buddemeyer-with-simulators

Finalidade

Projeta testes determinísticos com fakes, simuladores, replay e injeção de falhas para câmera, visão, relógio, NX102/CIP, robô e persistência.

Quando deve ser acionada. Qualquer feature ou correção que precise ser demonstrada sem a célula real; testes de FSM, timeout, reconexão, falha e restart.

Entradas e evidências. Interfaces reais dos adaptadores, efeitos observáveis, fixtures existentes, estados, eventos, tempos e dados sanitizados representativos.

Fluxo orientado pela skill

- Preferir interfaces pequenas e injeção de dependência a monkeypatch global.
- Controlar relógio, aleatoriedade, latência e ordem de eventos para eliminar flaky tests.
- Modelar freshness, cycle_id, ACK, timeout, erro, desconexão e cancelamento.
- Falhar com segurança se um endpoint real aparecer na configuração de teste.
- Separar unidade, componente Qt, integração simulada, contrato e HIL.
- Cobrir fluxo nominal, alvo ausente, baixa confiança, ACK duplicado, dado velho, recusa do robô e restart.

Saída esperada. Matriz requisito > teste, arquitetura dos doubles, comandos, resultados, cobertura relevante e lista do que ainda exige HIL.

Por que representa SOTA. Adota simulation-first e fault injection, permitindo testar mecanismos raros sem movimentar equipamento ou depender de disponibilidade da planta.

Exemplo de prompt

Crie fakes de câmera, NX102 e robô para validar [CENÁRIO]. Controle o relógio e injete ACK atrasado, timeout e reconexão. Proíba endpoints reais por padrão e informe explicitamente o que ainda exige HIL.

4.4. manage-buddemeyer-recipes-config

Finalidade

Trata receitas e configurações como interfaces versionadas, validadas e recuperáveis.

Quando deve ser acionada. Novo parâmetro, receita de produto, mudança de schema, unidade ou limite, importação/exportação, persistência ou migração.

Entradas e evidências. Formatos atuais, leitores, escritores, defaults, precedência, consumidores, versões, unidades, faixas, ambiente e amostras sanitizadas.

Fluxo orientado pela skill

- Definir schema explícito: versão, tipo, obrigatoriedade, faixa, enum e unidade.
- Validar na fronteira antes de publicar valores ao domínio ou hardware.
- Separar receita, ambiente, calibração, segredo e estado transitório.
- Gravar em temporário validado e substituir atômicamente quando suportado.
- Preservar última versão válida e detectar edição concorrente por revisão/hash.
- Não trocar receita no meio do ciclo sem contrato; usar snapshot por cycle_id.
- Migrar de forma determinística, idempotente, testável e com downgrade/backup.

Saída esperada. Schema antes/depois, consumidores, migração, testes de limite/interrupção/rollback, exemplo sem segredos e recuperação.

Por que representa SOTA. Eleva configuração a contrato de software, evitando defaults silenciosos, mistura de unidades e incompatibilidade entre versões.

Exemplo de prompt

Versione o schema das receitas e adicione [CAMPO]. Valide tipo, unidade e faixa; preserve a receita anterior se a escrita falhar; proíba troca fora de Idle e crie migração idempotente com rollback.

4.5. package-release-buddemeyer

Finalidade

Prepara build e release reproduzíveis com manifesto, dependências, testes, compatibilidade, implantação gradual e rollback.

Quando deve ser acionada. Criar instalador/pacote, preparar versão, documentar uma liberação ou planejar atualização do POC.

Entradas e evidências. Plataforma alvo, endpoint, gerenciador e lockfile, build atual, commit, Python/Qt, modelo, schema, calibração e revisão das interfaces/CLP.

Fluxo orientado pela skill

- Partir de árvore Git limpa e commit identificável.
- Executar lint, tipos e testes existentes com resultados registrados.
- Construir em ambiente limpo com dependências travadas.
- Gerar manifesto com versões, plataforma e hashes dos artefatos críticos.
- Comparar conteúdo final com allowlist e excluir segredos, dumps e caches.
- Executar smoke test do pacote com simuladores e restart.
- Definir bancada > simulação > HIL > piloto > produção e rollback do conjunto compatível.

Saída esperada. Artefato, manifesto, origem reproduzível, testes, matriz de compatibilidade, go/no-go, runbook e rollback testado.

Por que representa SOTA. Trata modelo, calibração, schema e aplicação como um conjunto versionado, reduzindo releases irreproduzíveis e rollback parcial.

Exemplo de prompt

Prepare um build reproduzível para [VERSÃO]. Gere manifesto com commit, Python/Qt, dependências, modelo, schema e calibração. Execute smoke test simulado e produza plano go/no-go e rollback. Não faça deploy.

4.6. incident-triage-buddemeyer

Finalidade

Porta de entrada para paradas, intermitências ou anomalias quando o domínio causal ainda não está demonstrado.

Quando deve ser acionada. Célula parou, degradou ou se comportou de modo anormal e a origem pode estar na aplicação, visão, calibração, NX102, rede, robô ou campo.

Entradas e evidências. Início/fim, turno, lote/receita, impacto, último ciclo bom, última mudança, logs, eventos, estados, contadores e relógios dos componentes.

Fluxo orientado pela skill

- Normalizar fuso e relógio entre aplicação, NX102, robô, câmera e rede.
- Coletar evidências somente leitura e reconstruir a linha do tempo.
- Separar sintoma, primeira falha causal e alarmes consequentes.
- Delimitar o domínio sem escolher culpado prematuramente.
- Formular no máximo três hipóteses com evidência favorável e contrária.
- Escolher para cada hipótese um teste discriminante de baixo risco.

- Propor mudança apenas depois de mecanismo causal plausível.

Saída esperada. Impacto, estado, timeline, evidências versus inferências, hipóteses ordenadas, próximo teste, mitigação, correção/rollback e RCA.

Por que representa SOTA. Aplica observabilidade causal e hipótese falsificável, evitando reinício por tentativa, blaming de rede ou edição sem reprodução.

Exemplo de prompt

A célula parou as [HORA]. A causa ainda pode ser visão, CLP ou rede.
Construa timeline somente leitura, limite-se a três hipóteses e indique o teste de menor risco que melhor diferencia cada mecanismo.

4.7. debug-nx102

Finalidade

Diagnostica eventos, tarefas, watchdogs, warm/cold start, memória, I/O, EtherCAT, lógica e FSM no Omron NX102/Sysmac.

Quando deve ser acionada. Há evidência específica no controlador: evento, watchdog, tarefa saturada, estado retido, erro de FB ou divergência de lógica.

Entradas e evidências. Modelo completo, unit version, firmware, Sysmac Studio, revisão implantada, tarefas/prioridades, módulos, estado, evento e sequência da falha.

Fluxo orientado pela skill

- Usar como autoridade os manuais da versão real confirmada.
- Correlacionar o primeiro evento causal, não apenas o último alarme.
- Verificar modo, reinício, watchdog, exceção e pico de tarefa.
- Revisar inicialização, retenção e warm/cold start.
- Procurar espera sem timeout, corrida, pulso perdido e escrita múltipla.
- Validar Done, Busy, Error e ErrorID, FSM, ACK e recuperação.
- Diferenciar CPU, NX bus, EtherCAT, EtherNet/IP, dispositivo e lógica.

Saída esperada. Evidências, causa provável, tarefa/trecho, teste de confirmação, correção mínima, rollback e estabilidade mensurável.

Por que representa SOTA. É orientada por versão e camada e explicita que comparar offline/online não autoriza sincronizar, forçar variável ou reiniciar o controlador.

Exemplo de prompt

O NX102 apresentou watchdog após warm start. Confirme versões, correlacione eventos e tarefas, revise retenção e FSM, e proponha apenas testes de leitura. Não transfira projeto, mude modo ou force variável.

4.8. debug-cip-ethernet-ip

Finalidade

Investiga perda, timeout, reconexão e dado obsoleto em CIP sobre EtherNet/IP entre aplicação, NX102 e dispositivos.

Quando deve ser acionada. Timeout de comunicação, tag data link instável, status CIP, reconexão ou valor inconsistente com evidência de camada de comunicação.

Entradas e evidências. Origem/destino, modelo/firmware, IP e porta, caminho CIP, I/O implícito ou mensagem explícita, produtor/consumidor, tamanho, RPI, timeout e timestamps.

Fluxo orientado pela skill

- Separar aplicação/freshness, CIP, IP/Ethernet, switch/carga e físico.
- Interpretar status geral/estendido e caminho, sem concluir 'rede' apenas por timeout.
- Comparar baseline e falha: perda, jitter, link flap, descartes e reconexões.
- Avaliar RPI e carga apenas contra limites documentados da versão real.
- Preferir espelhamento/TAP e estatísticas a scans agressivos.
- Correlacionar captura e logs por timestamp e cycle_id quando disponível.

Saída esperada. Topologia, conexão afetada, timeline, baseline versus falha, hipótese, teste discriminante, correção reversível e validação sob carga.

Por que representa SOTA. Une diagnóstico de protocolo e freshness da aplicação, reduzindo falsos diagnósticos e mudanças prematuras de RPI, QoS ou topologia.

Exemplo de prompt

Após reconexão CIP, o supervisor consumiu coordenada antiga. Diferencie sessão, rede, endpoint e política de freshness. Correlacione status e timestamps; não altere RPI ou switch sem baseline e janela.

4.9. harden-buddemeyer-24x7

Finalidade

Faz revisão preventiva de confiabilidade, observabilidade, recuperação, deploy e operação contínua.

Quando deve ser acionada. Objetivo de robustecer o sistema, criar backlog 24x7, definir SLO/RTO/RPO, reduzir ponto único ou ensaiar recuperação. Não é a primeira skill de incidente desconhecido.

Entradas e evidências. Dependências, interfaces, volume, ciclo nominal/pico, falhas históricas, MTBF/MTTR, deploy, rollback, backup, janelas e objetivos aprovados.

Fluxo orientado pela skill

- Modelar fluxo ponta a ponta e pontos únicos de falha.
- Revisar timeout, cancelamento, retry limitado com backoff, idempotência e circuit breaker.
- Limitar filas/memória e impedir reuso de dados antigos em reconexão.
- Definir estado seguro para perda de câmera, modelo, CLP, robô, rede, disco e energia.
- Separar liveness, readiness e saúde de dependências.
- Estruturar logs, métricas, dashboards, alertas e runbook.
- Ensaiar falhas em bancada e validar rollback sob carga representativa.

Saída esperada. Matriz de riscos, backlog risco-esforço-reversibilidade, responsáveis, testes, dashboards, alertas, runbook e riscos aceitos.

Por que representa SOTA. Aplica reliability engineering orientada a degradação segura e recuperação ensaiada, sem prometer ausência total de falhas ou inventar SLOs.

Exemplo de prompt

Avalie o repositório para operação 24x7. Mapeie pontos únicos, timeouts, retries, filas, freshness, métricas e rollback. Gere backlog priorizado com evidência, proprietário, teste e critério de sucesso.

4.10. validate-pick-place-cycle

Finalidade

Valida o ciclo ponta a ponta e o handshake aplicação - NX102 - robô como uma transação identificável.

Quando deve ser acionada. Estado preso, comando duplicado, pulso perdido, ACK ambíguo, timeout, restart em ciclo ou alteração da sequência ACK, Pick, Place e Complete.

Entradas e evidências. FSM real, produtor/consumidor de cada sinal, nível ou pulso, entrada/saída, timeout, erro, cancelamento, identidade do ciclo e comportamento em reconexão.

Fluxo orientado pela skill

- Desenhar a FSM observada e comparar com o contrato versionado.
- Usar cycle_id ou contador correlacionável entre detecção, coordenada e movimento.
- Garantir ACK inequívoco e repetição idempotente sem movimento duplicado.
- Rejeitar comando, frame ou coordenada obsoletos por estado, idade e identidade.
- Encontrar estado sem saída, timeout ausente, race, escrita concorrente e pulso curto.
- Definir erro, cancelamento e retorno seguro a Idle sem pular permissivos.
- Testar happy path, falhas, timeout, restart por estado e ACK repetido.

Saída esperada. FSM antes/depois, invariantes, falha, mudança mínima, testes por transição, observação, risco e rollback.

Por que representa SOTA. Usa semântica transacional, identidade e idempotência para reduzir a classe mais perigosa de bug: repetir comando lógico como movimento físico.

Exemplo de prompt

Revise ACK > Pick > Place > Complete. Identifique estados sem timeout, pulsos perdidos e risco de duplicidade. Garanta cycle_id, freshness, recuperação após restart e testes simulados por transição.

4.11. validate-vision-inference

Finalidade

Avalia toda a cadeia de visão: aquisição, pré-processamento, modelo/checkpoint, inferência, pós-processamento, seleção e publicação da coordenada.

Quando deve ser acionada. Alvo ausente ou incorreto, confiança inadequada, frame velho, latência, orientação/resolução ou divergência entre detecção e seleção final.

Entradas e evidências. Câmera/resolução, iluminação, hash do modelo/checkpoint, dispositivo, dependências, classes, limiar, NMS, regra de seleção, latência e conjunto de teste.

Fluxo orientado pela skill

- Confirmar que artefatos pertencem ao mesmo checkpoint/configuração.
- Validar orientação, cor, normalização, resize/letterbox e coordenadas no frame original.
- Comparar detecção bruta, pós-processamento e alvo final.
- Medir precision/recall e erro operacional por variedade, posição, oclusão e iluminação.
- Impedir reuso de frame/detecção após restart ou reconexão.
- Registrar imagem, predição, confiança, bbox/máscara, timestamp, cycle_id e motivo de rejeição.

Saída esperada. Estágio causal, amostras reproduzíveis, métricas antes/depois, alteração, teste offline, impacto e rollback.

Por que representa SOTA. Separa erro do modelo de erro de regra, adiciona provenance temporal e mede desempenho por cenário, em vez de confiar em exemplos anedóticos.

Exemplo de prompt

O modelo detectou três embalagens, mas o alvo publicado estava errado. Compare detecção bruta, pós-processamento e seleção final; preserve frame, confiança, coordenada e cycle_id; teste offline e quantifique.

4.12. validate-camera-robot-calibration

Finalidade

Valida a transformação entre pixel, plano de trabalho, referencial do robô, ferramenta e ponto de pega.

Quando deve ser acionada. Erro pixel-mm, offset, escala, rotação, deslocamento nos cantos, mudança de câmera/poste, foco, resolução, altura ou TCP.

Entradas e evidências. Resolução/crop, parâmetros da câmera, posição e foco, referenciais, pares de pontos, unidade, modelo matemático, versão da calibração e tolerância aprovada.

Fluxo orientado pela skill

- Identificar escala/offset, transformação afim, homografia ou calibração completa.
- Confirmar eixos, origem, sinal, unidade, rotação e ordem das transformações.
- Separar pontos de ajuste e validação.
- Calcular resíduos, RMSE, percentis e erro máximo em milímetros.
- Testar centro, cantos, bordas e alturas reais; quantificar erro da hipótese planar.
- Verificar distorção, resolução/crop, movimento da câmera, folga e TCP.
- Versionar matriz, dados, responsável, configuração e código consumidor.

Saída esperada. Modelo observado, dados válidos, mapa de erro, regiões não confiáveis, causa, tolerância, teste físico controlado e rollback.

Por que representa SOTA. Usa validação fora da amostra e distribuição espacial do erro, evitando aceitar uma calibração que só funciona nos pontos usados para ajustá-la.

Exemplo de prompt

O ponto está correto no centro e desloca 12 mm nos cantos. Identifique o modelo geométrico, separe pontos de ajuste/validação, calcule RMSE e erro máximo por região e proponha teste físico controlado e rollback.

4.13. debug-startup-restart

Finalidade

Diagnostica sistemas que funcionam no primeiro Start e falham depois de Stop/Start, procurando recursos e estados residuais.

Quando deve ser acionada. Câmera não retorna, porta fica ocupada, callback duplica, thread permanece, fila antiga reaparece ou FSM reinicia parcialmente.

Entradas e evidências. Sequência exata, logs das duas inicializações, recursos adquiridos/liberados, threads/processos, câmera, socket/CIP, timers, callbacks, flags e estado da FSM.

Fluxo orientado pela skill

- Comparar primeira inicialização, parada e segunda inicialização como fases observáveis.
- Inventariar câmera, socket, porta, worker, executor, arquivo, GPU, modelo e callbacks.
- Verificar ordem e idempotência de start, stop, close, join e cancelamento.
- Procurar singleton/global residual, flag, fila velha, zombie, callback duplicado e sessão expirada.
- Exigir timeout finito e visibilidade para falha de cleanup.
- Testar restart em Idle e estados relevantes, inclusive dependência indisponível.
- Repetir ciclos e observar handles, threads, memória e conexões.

Saída esperada. Reprodução, diferença entre execuções, recurso causal, correção mínima, teste automatizado, rollback e evidência de múltiplos restarts.

Por que representa SOTA. Transforma um bug intermitente de lifecycle em experimento repetível, com métricas de recursos e teste de regressão que evita o clássico 'reiniciar resolve'.

Exemplo de prompt

A primeira abertura funciona; após Stop/Start a câmera não envia frames. Compare as três fases, liste recursos e callbacks, reproduza sem hardware e aceite somente após 20 ciclos sem crescimento de threads ou dado velho.

5. As 3 rules

Rules mantêm invariantes próximos ao código. Elas devem ser curtas o suficiente para não dominar o contexto e fortes o suficiente para impedir atalhos recorrentes.

5.1 00-poc-context-and-safety.mdc

Modo

alwaysApply: true - presente em toda conversa do projeto.

- Trata o sistema como uma célula integrada: visão, transformação, supervisorio, NX102, CIP, robô e efetor.
- Exige leitura dos documentos do projeto e testes relevantes antes de editar.
- Separa fato observado, inferência e dado não verificado.
- Proíbe inventar tag, IP, assembly, firmware, offset, tolerância ou estado.
- Determina triagem somente leitura para incidentes sem causa.
- Veta forçar I/O, burlar permissivos e alterar equipamento sem autorização, backup, janela e rollback.
- Exige mudanças pequenas, reversíveis, observáveis e com teste de confirmação.

5.2 pyside-ui-architecture.mdc

Modo

alwaysApply: false com descrição - selecionada quando o contexto envolve PySide6/Qt.

- Preserva apresentação > aplicação > domínio/FSM > adaptadores.
- Impede I/O industrial direto em widgets e bloqueio do event loop.
- Restringe atualização visual a thread da UI por sinais/slots.
- Exige ownership, cancelamento e cleanup de workers, timers e callbacks.
- Protege arquivos gerados e exige testes de lifecycle sem hardware.

5.3 hardware-adapters.mdc

Modo

alwaysApply: false com descrição - selecionada para câmera, visão, NX102/CIP, robô e campo.

- Encapsula SDKs, sockets e protocolos atrás de interfaces estreitas.
- Modela configurar, conectar, ready, operar, degradar, desconectar e fechar.
- Exige timeout, cancelamento, erro contextual e retry somente quando idempotente.
- Transporta qualidade, timestamp/idade e cycle_id quando aplicável.
- Impede que reconexão transforme frame, ACK ou coordenada velha em dado novo.
- Veta bypass de safety e exige testes primeiro com fake/simulador.

5.4 Rule versus skill

Pergunta	Use rule	Use skill
Deve valer sempre ou por tipo de arquivo?	Sim	Não necessariamente
É um workflow com entradas, passos e saída?	Não é o foco	Sim
Precisa ser chamada por /nome?	Normalmente não	Pode ser
Deve carregar muito contexto?	Não	Somente quando relevante
Exemplo no POC	Não bloquear UI	Diagnosticar restart

6. Hooks e controles determinísticos

Princípio

Use LLM para julgamento contextual e código determinístico para invariantes verificáveis. Hooks são uma camada local de feedback e governança; CI continua necessário.

6.1 guard-industrial-commands.py

O hook roda antes de comandos de shell que combinam com o matcher do manifesto. Ele recebe o comando em JSON e devolve uma decisão.

Classe	Exemplo	Decisão
Leitura/teste	pytest -q; git status	allow
Destrutivo	git push --force; rm recursivo; rsync --delete	ask
Rede mutável	ip link set; nmcli modify; iptables	ask
Industrial mutável	write_plc.py com alvo nx102; deploy robot controller	ask
Entrada vazia/inválida	JSON sem command	deny

- O manifesto aplica timeout curto e failClosed: true para a operação avaliada.
- O script reconhece comandos encadeados para evitar que um prefixo de leitura esconda uma ação posterior.
- Regex pode produzir falso positivo ou falso negativo; ferramentas específicas do POC devem ser adicionadas em PR.
- A resposta ask exige que o humano confirme alvo, backup, janela, sucesso e rollback.

6.2 validate-edited-file.py

Executado depois de uma edição feita pelo Agent. O check é rápido, local e sem dependências externas:

- Python: parseia a AST, sem gerar __pycache__.
- JSON: faz parse integral.
- TOML: usa tomlib quando disponível.
- SKILL.md: exige frontmatter e os campos name e description.
- Rules .mdc: exige frontmatter e alwaysApply.
- Recusa validar arquivo fora das raízes declaradas do workspace.

O que o hook não faz

Não executa lint, type-check, testes funcionais, análise de concorrência ou validação de schema industrial. Ele detecta erros objetivos imediatos; os demais checks pertencem ao repositório e ao CI.

6.3 Testes manuais dos hooks

```
printf '%s\n' '{"command":"pytest -q","workspace_roots":[]}' \
| python3 .cursor/hooks/guard-industrial-commands.py

printf '%s\n' '{"command":"deploy robot controller","workspace_roots":[]}' \
| python3 .cursor/hooks/guard-industrial-commands.py

python3 -m json.tool .cursor/hooks.json >/dev/null
```

7. Subagente industrial-change-verifier

O verificador foi criado para revisão independente depois da implementação. Sua descrição diz ao Agent quando delegar; seu frontmatter fixa model: inherit e readonly: true.

7.1 O que ele verifica

- Objetivo, critérios de aceite, não objetivos e aderência do diff ao escopo.
- Separação UI, aplicação, domínio/FSM, adaptadores e persistência.
- Event loop, races, callbacks duplicados, cleanup, timeouts e retry idempotente.
- cycle_id, ACK, freshness, restart, falha segura e ausência de bypass.
- Schema, unidades, migração, atomicidade, compatibilidade e rollback.
- Release reproduzível, manifesto, exclusão de segredos e smoke test.
- Se os testes exercitam o mecanismo alterado, e não apenas linhas periféricas.

7.2 Severidade e decisão

Nível	Definição
Bloqueador	Risco de segurança/integridade ou critério essencial não atendido.
Alto	Regressão provável, perda de dados, deadlock/race ou ausência de recuperação.
Médio	Robustez, manutenção, cobertura ou observabilidade insuficiente.
Baixo	Melhoria localizada sem impacto imediato.

O relatório termina com APROVADO, APROVADO COM RESSALVAS ou NÃO APROVADO, testes observados, não executados, riscos residuais e o que ainda exige HIL/POC.

7.3 Prompt de revisão

Revise esta mudança usando industrial-change-verifier em modo somente leitura. Compare pedido, critérios e diff; verifique arquitetura, threads, lifecycle, FSM, freshness, idempotência, configuração, testes, segredos e rollback. Não edite. Classifique achados por severidade e finalize com APROVADO, APROVADO COM RESSALVAS ou NÃO APROVADO. Liste tudo que ainda exige HIL ou validação local.

8. Documentos que tornam o Cursor especialista

Skills ensinam como raciocinar; documentos do projeto fornecem os fatos locais. A combinação evita instruções genéricas e alucinações sobre o POC.

Documento	Conteúdo	Responsável pela atualização
POC_CURITIBA_CONTEXT.md	Objetivo, componentes, versões, topologia, convenções, fonte e premissas.	Líder técnico e especialistas de cada componente.
INTERFACES_AND_STATE_MACHINE.md	Tags/sinais, produtor, consumidor, tipo, unidade, validade, estado seguro, estados e transições.	Automação/CLP, robô e aplicação em conjunto.
CODEBASE_MAP.md	Paths, endpoints, classes, interfaces, testes, build, arquivos gerados e ambientes.	Equipe de software; revisão por PR.

8.1 Regra de qualidade da informação

- Substituir 'A confirmar' somente com fonte verificável: código, export Sysmac, manual da versão real, desenho aprovado ou teste registrado.
- Registrar data e revisão da fonte.
- Não armazenar senhas, tokens, chaves, dumps sensíveis ou dados industriais desnecessários.
- Atualizar via branch e PR para que o conhecimento tenha histórico e revisão.
- Depois que os paths estabilizarem, avaliar paths/globs específicos para reduzir ainda mais o contexto.

9. Instalação e atualização no POC

9.1 Pré-requisitos

- Cursor atualizado com Agent Skills, Rules, Hooks e Subagents.
- Raiz real do repositório aberta no Cursor.
- Git funcional e permissão para branch/PR.
- Python 3 disponível como python3 para os hooks; no Windows, ajustar para py -3 se necessário.
- Mudanças locais existentes preservadas antes da extração.

9.2 Passo a passo

- No terminal integrado, execute `git rev-parse --show-toplevel` e confirme a raiz.
- Revise `git status --short`. Faça commit ou preserve alterações pelo processo da equipe.
- Crie uma branch exclusiva, por exemplo `chore/cursor-sota-poc-curitiba`.
- Copie `cursor-sota-buddemeyer-poc-curitiba.zip` para a raiz.
- Use `unzip -l` para confirmar que os caminhos começam por `.cursor/`, sem pasta intermediária.
- Extraia com `unzip -o ... -d .` e aplique `chmod +x .cursor/hooks/*.py` no macOS/Linux.
- Confirme 13 SKILL.md, 3 .mdc, 1 agente e 2 scripts de hook.
- Marque o workspace como confiável para permitir hooks do projeto.
- Abra nova conversa Agent ou use Developer: Reload Window.
- Em Customize > Skills, confirme as 13 skills em Agent Decides.
- Preencha os três documentos de docs/project/ com fatos confirmados.

- Revise o diff, commit, push e abra PR antes de incorporar ao ramo principal.

9.3 Comandos de instalação

```
git rev-parse --show-toplevel
git status --short
git switch -c chore/cursor-sota-poc-curitiba

unzip -l cursor-sota-buddemeyer-poc-curitiba.zip
unzip -o cursor-sota-buddemeyer-poc-curitiba.zip -d .
chmod +x .cursor/hooks/*.py

find .cursor/skills -name SKILL.md | wc -l # esperado: 13
find .cursor/rules -name '*.mdc' | wc -l # esperado: 3
python3 -m json.tool .cursor/hooks.json >/dev/null
git diff --check
```

macOS

A pasta `.cursor` começa por ponto e fica oculta no Finder. Pressione `Command + Shift + .` para exibi-la. O Cursor a reconhece mesmo quando o Finder não a mostra.

9.4 Estrutura no local errado

Correto: `<repo>/cursor/skills/...` Incorreto:

`<repo>/cursor-sota-buddemeyer-poc-curitiba/.cursor/...` No segundo caso, o wrapper deve ser removido e o conteúdo movido um nível acima.

10. Prompts estruturados para o chat

O prompt não deve dizer apenas 'faça'. Ele deve estabelecer uma função de verificação: como saber que a mudança atende ao pedido e que não criou risco novo.

10.1 Bloco comum

MODO DE TRABALHO

- Comece em Plan. Inspeccione arquivos, classes, chamadores, contratos e testes realmente encontrados.
- Consulte os documentos em docs/project/ quando relevantes.
- Separe fatos, inferências e itens não verificados. Não invente paths, tags, IPs, versões, estados ou comandos.
- Preserve mudanças locais e não refatore fora do escopo.
- Não acesse hardware, force I/O ou altere CLP, robô, safety, firmware ou rede sem autorização explícita.
- Valide primeiro com unidade e simuladores; marque o que exige HIL.
- Após implementar, use industrial-change-verifier.

ENTREGA

- Comportamento antes/depois e arquivos alterados.
- Testes executados com resultado literal.
- Riscos residuais, itens não verificados e rollback.

10.2 Prompt para adicionar feature

OBJETIVO

Adicione [FEATURE] ao supervisor para que [USUÁRIO/PROCESSO] consiga [RESULTADO OPERACIONAL].

CONTEXTO CONFIRMADO

- Fluxo atual: [COMO FUNCIONA].
- Ponto de entrada: [ARQUIVO/CLASSE ou NÃO VERIFICADO].
- Estados envolvidos: [FSM].
- Dependências: [UI/VISÃO/CALIBRAÇÃO/NX102/ROBÔ/RECEITA].
- Evidências: [ISSUE/LOG/DOCUMENTO/COMMIT].

CRITÉRIOS DE ACEITE

- Dado [CONDIÇÃO], quando [AÇÃO], então [RESULTADO MENSURÁVEL].
- Em [ERRO], o sistema deve [FALHA SEGURA/MENSAGEM].
- A UI permanece responsiva.
- Stop/Start não duplica callback, thread, comando ou dado antigo.
- Compatibilidade preservada com [CONTRATO].

NÃO OBJETIVOS

- Não alterar [FORA DO ESCOPO].

EXECUÇÃO

1. Comece em Plan e apresente mapa de impacto.
2. Proponha a menor solução reversível.
3. Separe UI, aplicação, domínio/FSM e adaptadores.
4. Crie testes para cada critério.
5. Execute unidade, Qt e integração simulada aplicáveis.
6. Use industrial-change-verifier e entregue riscos/rollback.

Exemplo

Seleção de receita

Adicionar tela que permite trocar receita apenas em Idle, mostra identificador/revisão, valida antes de ativar, mantém a anterior em erro e registra a revisão com cycle_id. Testar receita inválida, troca durante Pick/Place, escrita interrompida e abertura repetida sem sinais duplicados.

10.3 Prompt para customizar função

OBJETIVO

Customize [FUNÇÃO/COMPORTAMENTO] para [NOVO RESULTADO], preservando [ASSINATURA, UNIDADES, EXCEÇÕES E EFEITOS].

LOCALIZAÇÃO E CONTRATO

- Referência inicial: [PATH ou DESCONHECIDO].
- Chamadores: [LISTA ou DESCONHECIDO].
- Entradas/saídas: [TIPOS E UNIDADES].
- Thread: [UI/WORKER/ASYNC/DESCONHECIDO].

NOVA REGRA

- [REGRA DETERMINÍSTICA].
- Em [ERRO], deve [COMPORTAMENTO SEGURO].

EXECUÇÃO

1. Localize definições, chamadas, overrides, sinais e testes.
2. Explique o comportamento atual com evidência de código.
3. Mostre impacto de compatibilidade antes de editar.
4. Faça o menor diff e preserve APIs sem necessidade de mudança.
5. Adicione regressão, limites, inválidos, timeout e restart.
6. Execute checks e industrial-change-verifier.

Exemplo

Seleção do alvo

Localizar a função real, rejeitar detecção cuja idade exceda o limite configurado, preservar a política atual e aplicar desempate determinístico. Nunca devolver coordenada default quando não houver alvo. Testar frame atual/obsoleto, empate, vazio e restart com fila antiga.

10.4 Prompt para resolver problema

INCIDENTE

- Sintoma observável: [SEM INTERPRETAÇÃO].
- Início/frequência: [HORA/FREQUÊNCIA].
- Último ciclo bom: [ID/HORA].
- Ambiente, versão e commit: [CONFIRMADOS].
- Última mudança: [COMMIT/CONFIG/MANUTENÇÃO].

REPRODUÇÃO E EVIDÊNCIAS

1. [ESTADO INICIAL]
 2. [AÇÃO]
 3. [RESULTADO ATUAL]
 4. Esperado: [RESULTADO]
- Logs/eventos/testes tentados: [REFERÊNCIAS]

LIMITES

- Comece somente leitura.
- Não reinicie, limpe erro, force I/O ou mude configuração para testar.
- No máximo três hipóteses; para cada uma, evidência favorável, contrária e teste discriminante de baixo risco.

EXECUÇÃO

1. Construa timeline com relógios/fusos explícitos.
2. Identifique a primeira falha causal.
3. Reproduza em teste ou simulador.
4. Somente então implemente a menor correção.
5. Adicione teste que falha antes e passa depois.
6. Rode regressão, restart quando aplicável e o verificador.

ACEITE: [MÉTRICA, CICLOS OU PERÍODO SEM FALHA].

Exemplo

Falha no segundo Start

Comparar primeira inicialização, parada e segunda inicialização. Inventariar câmera, threads, timers, callbacks, filas e conexões. Aceitar somente após 20 ciclos simulados sem aumento de recursos, duplicidade ou reaproveitamento de frame/cycle_id.

11. Roteamento automático esperado

Teste cada frase em uma conversa nova e sem escrever o nome da skill. O roteamento é semântico e pode combinar competências.

Prompt	Skill esperada
Adicione uma tela de gestão de receitas com validação e rollback.	customize-buddemeyer-supervisory + manage-buddemeyer-recipes-config
A tela congela enquanto a inferência roda.	develop-pyside6-supervisory-ui
Teste timeout e ACK duplicado sem o NX102 real.	test-buddemeyer-with-simulators
Prepare build reproduzível e rollback.	package-release-buddemeyer
A célula parou e a causa é desconhecida.	incident-triage-buddemeyer
O NX102 teve watchdog após warm start.	debug-nx102
Há timeout CIP e dado antigo após reconexão.	debug-cip-ethernet-ip
Crie backlog de robustez para 24x7.	harden-buddemeyer-24x7
Revise ACK, Pick, Place e Complete.	validate-pick-place-cycle

Prompt	Skill esperada
O preditor escolheu o alvo errado.	validate-vision-inference
O erro em milímetros cresce nos cantos.	validate-camera-robot-calibration
Funciona no primeiro Start e falha no segundo.	debug-startup-restart
Customize o comportamento da função de seleção.	customize-buddemeyer-supervisory, combinada com a skill do domínio
Se o roteamento falhar Refine a descrição da skill por evidência e teste novamente. Para seleção determinística em uma tarefa, use /nome-da-skill. Não duplique skills genéricas apenas para forçar o Agent.	

12. Por que esta arquitetura está no SOTA

SOTA aqui significa alinhamento com o estado atual das capacidades do Cursor e com práticas modernas de engenharia de agentes e software industrial. Não significa certificação funcional, perfeição ou ausência de risco.

Princípio SOTA	Como foi implementado	Risco reduzido
Divulgação progressiva	Skills pequenas, descrições discriminantes e conteúdo carregado sob demanda.	Contexto excessivo e conflito de instruções.
Defesa em profundidade	Rules + skills + testes + hooks + verificador + PR.	Dependência de uma única camada falível.
Determinismo onde possível	AST, JSON, TOML, frontmatter e gate de shell em scripts.	Erros objetivos deixados ao julgamento do LLM.
Simulation-first	Fakes, relógio controlado, fault injection e separação do HIL.	Testar diretamente na célula e baixa reprodutibilidade.
Revisão independente	Subagente readonly com relatório por severidade.	Viés de confirmação de quem implementou.
Contratos e provenance	cycle_id, timestamp, qualidade, versão, schema, modelo e calibração.	Dado antigo, incompatibilidade e diagnóstico sem rastreio.
Arquitetura hexagonal	UI e domínio isolados de SDKs e protocolos por adaptadores.	Acoplamento a hardware e testes impossíveis.
Release reproduzível	Commit, lockfile, manifesto, hash, compatibilidade e rollback conjunto.	Build irreproduzível e rollback parcial.
Governança versionada	Tudo no repositório, revisado por branch e PR.	Conhecimento local invisível e mudança sem auditoria.
Humano no circuito	ask para comandos suspeitos, HIL separado e aprovação local.	Automação indevida de ação física.

12.1 Evolução do pacote

Estágio	Cobertura	Lacuna
Original	Triagem, NX102, CIP e hardening.	Duplicatas, resíduos e ausência dos domínios centrais.
Primeira revisão	Mais visão, calibração, ciclo e restart: 8 skills.	Forte em diagnóstico, incompleto para customizar o app.
Versão SOTA	13 skills, 3 rules, 2 hooks, verificador, prompts e docs de contexto.	Aprimorar continuamente paths, CI e dados reais do POC.

12.2 O que ainda falta no repositório real

- Preencher versões, firmware, biblioteca CIP e paths efetivos.
- Confirmar contrato das tags, RPI, timeouts, estados e proprietários.
- Implementar ou consolidar simuladores reais nas interfaces existentes.
- Registrar comandos oficiais de lint, type-check, testes e build em CODEBASE_MAP.md.
- Espelhar checks essenciais no CI para não depender apenas do Cursor local.
- Definir critérios aprovados de HIL, tolerância espacial, tempo de ciclo e rollout.

13. Governança e fluxo diário

13.1 Fluxo recomendado

- Abrir branch e confirmar a árvore de trabalho.
- Descrever objetivo, aceite, não objetivos e limites.
- Pedir Plan e mapa de impacto antes de edição relevante.
- Confirmar decisões arquiteturais ou dados bloqueantes.
- Implementar o menor incremento reversível.
- Executar unidade, Qt e integração simulada aplicáveis.
- Rodar lint, tipos, testes e build oficiais do repositório.
- Acionar industrial-change-verifier em modo readonly.
- Corrigir bloqueadores e achados altos dentro do escopo.
- Revisar diff, riscos, rollback e pendências de HIL.
- Abrir PR e obter revisão técnica humana.
- Executar bancada/HIL/POC somente sob procedimento local.

13.2 Critérios de aceite da estrutura

- .cursor existe diretamente na raiz do repositório.
- 13 skills aparecem em Customize > Skills > Agent Decides e por /.
- 3 rules são reconhecidas e o verificador está disponível como readonly.
- O gate retorna allow para teste e ask para exemplo industrial.
- O validador retorna {} para arquivo válido.
- Os documentos do projeto possuem responsáveis para os itens pendentes.
- Os prompts de feature, função e problema foram ensaiados.
- A mudança passou por PR e nenhuma ação online foi executada pela instalação.

13.3 Manutenção

- Medir quais prompts acionam cada skill e refinar sobreposições.
- Adicionar novos adaptadores/comandos ao gate por casos reais, não por suposição.
- Revisar o pacote após mudança relevante no Cursor, PySide6, arquitetura ou release.
- Não proliferar skills genéricas; crie uma nova apenas quando houver responsabilidade e gatilho distintos.
- Versionar toda alteração de rule, skill, hook ou subagente como código de produção.

14. Glossário técnico

Termo	Significado no POC
ACK	Acknowledge. Confirmação explícita de recebimento/aceitação; não deve ser confundida com conclusão física.
FSM	Finite State Machine. Modelo de estados e transições, com entradas, saídas, timeouts, erro e recuperação.
cycle_id	Identificador correlacionável de uma tentativa de ciclo entre frame, coordenada, pick, place e complete.
Freshness	Validade temporal do dado, derivada de timestamp, idade, sequência, qualidade e estado.
Idempotência	Repetir uma solicitação não produz efeito físico duplicado além do resultado previsto.
Timeout	Limite finito de espera com transição/erro definido; evita estado preso indefinidamente.
Retry	Nova tentativa controlada, limitada e observável. Só é segura quando a operação é idempotente ou reconciliada.
Circuit breaker	Mecanismo que interrompe tentativas repetidas contra dependência falha e permite recuperação controlada.
Smoke test	Teste curto que confirma que o artefato inicia, fica pronto, executa fluxo básico e encerra corretamente.
HIL	Hardware-in-the-loop. Validação controlada com hardware ou controlador real/representativo.
SLO	Objetivo mensurável de nível de serviço, como disponibilidade ou latência, sujeito a aprovação.
RTO/RPO	Tempo alvo de recuperação e ponto alvo de recuperação de dados/configuração.
Rollback	Procedimento testado para restaurar um conjunto compatível anterior, não apenas um binário isolado.
Provenance	Registro da origem e versão de código, modelo, calibração, receita, dado e decisão.
RPI	Requested Packet Interval em EtherNet/IP; deve ser ajustado apenas com baseline e documentação da versão real.
QSignalSpy	Ferramenta Qt para observar e verificar emissões de sinais em testes.

15. Referências oficiais

[1] Cursor Agent Skills

<https://cursor.com/docs/skills>

Descoberta em .cursor/skills, pastas aninhadas, descrição, invocação automática e manual.

[2] Cursor Project Rules

<https://cursor.com/docs/rules>

Rules .mdc, alwaysApply, aplicação inteligente, globs e escopo de projeto.

[3] Cursor Hooks

<https://cursor.com/docs/hooks>

hooks.json, eventos, JSON por stdin/stdout, permission e failClosed.

[4] Cursor Subagents

<https://cursor.com/docs/subagents>

Subagentes customizados, descrição, model e readonly.

[5] Cursor Agent Security

<https://cursor.com/docs/agent/security>

Modelo de permissões, risco de ferramentas e boas práticas de segurança.

[6] Agent Skills Specification

<https://agentskills.io/specification>

Formato interoperável e princípios de skills.

[7] Qt for Python - Signals and Slots

https://doc.qt.io/qtforpython-6/tutorials/basictutorial/signals_and_slots.html

Sinais, slots e comunicação entre objetos Qt.

[8] Qt for Python - QSignalSpy

<https://doc.qt.io/qtforpython-6/PySide6/QtTest/QSignalSpy.html>

Observação de emissões de sinais em testes.

Nota de atualização

As capacidades do Cursor evoluem. Este documento reflete a documentação oficial consultada em 8 de setembro de 2026. Revalidar formato e comportamento de hooks, skills e subagentes em atualizações futuras do IDE.

DHS Data Heurística Solutions | Manual técnico do POC Buddemeyer - Curitiba

Documento de engenharia. Validação humana e procedimentos locais permanecem obrigatórios.